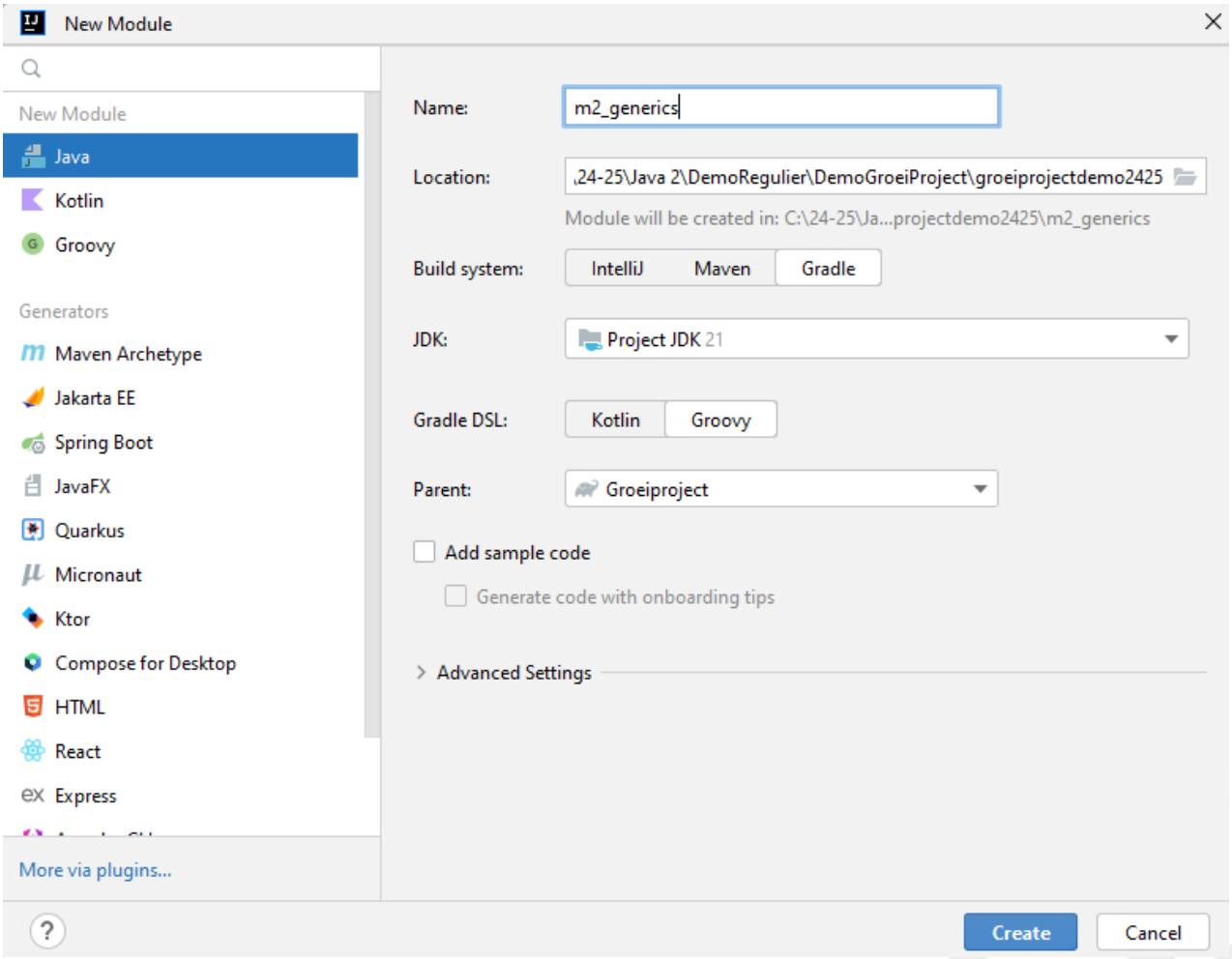
GROEIPROJECT
MODULE 2: GENERICS EN DOCUMENTEREN



1. Voorbereiding

1.1. Open het groeiproject en maak daarin een tweede module: `m2_generics` (*File* > *New* > *Module*):
Als je twijfelt hoe dat moet: zie **stappenplan_git_Gradle.pdf**



1.2. Controleer de `build.gradle`: daar moet je een aantal zaken aanpassen en de `mainClassName` veranderen naar `Demo_2`.

1.3. Kopieer vanuit de vorige module de hele package `be.kdg...model` (alle klassen behalve de multiklasse) en de package `be.kdg...data` naar de `java` map van de nieuwe module.
De multiklasse `Dicatators` zullen we deze week niet nodig hebben!

2. Generics

We gaan zelf onze eigen generieke datastructuur bouwen met de naam "**PriorityQueue**". Deze klasse bestaat eigenlijk al in de Java JDK (`java.util.PriorityQueue`), maar ze werkt anders dan we zouden willen, dus we maken deze klasse opnieuw zelf *from scratch*!

Werking:

- **enqueue**: Elk element dat aan de queue wordt toegevoegd krijgt een bepaalde prioriteit mee (cijfer van 1..5) en wordt volgens die prioriteit (hoogste eerst) in de queue gestockeerd. Elementen met dezelfde prioriteit worden in hun reeks volgens het FIFO-principe toegevoegd.
- **dequeue**: In de reeks met de hoogste prioriteit wordt het oudste element verwijderd (FIFO).

2.1. Maak onder een nieuwe package `generics` de generieke interface `FIFOQueue`. Neem de volgende code gewoon over:

```
public interface FIFOQueue<T> {

    boolean enqueue(T element, int priority);

    T dequeue();

    int search(T element);

    int getSize();
```

2.2. Maak in dezelfde package `generics` de klasse `PriorityQueue` die bovenstaande interface implementeert. Opgelet! Deze klasse moet generiek zijn voor elk Object-type; gebruik dus <T>

2.3. We zullen als onderliggende implementatie werken met een `TreeMap`: de **key** is het prioriteitsgetal (1..5) en de **value** is een `LinkedList` van elementen met dezelfde prioriteit.
Declareer deze `TreeMap` als private attribuut en zorg ervoor dat de keys in dalende volgorde staan gesorteerd (TIP: `Comparator.reverseOrder`)

2.4. Implementeer nu alle methoden van de interface `FIFOQueue`:

- **enqueue:** Het element en de prioriteit komen als parameters binnen. Onderzoek eerst of het element nog niet voorkomt in de map (ongeacht de prioriteit). Indien **wel**, dan retourneer je false.
  Indien **niet**, dan kijk je of er al een entry bestaat van deze prioriteit. Als dat het geval is, dan voeg je het nieuwe element achteraan toe in deze prioriteitsreeks, anders maak je een nieuwe List met daarin dit nieuwe element.
- **dequeue**: Retourneer het element dat zich **vooraan** bevindt in de hoogste prioriteitsreeks en verwijder het uit de queue.
- **search**: Zoek het element op in de map. Indien gevonden, dan retourneer je de positie **in de volledige queue** (dus niet de positie in de LinkedList). Indien niet gevonden: `-1`
- **getSize**: Geef het totaal aantal elementen in de volledige queue.
- **toString**: Genereer een overzicht van de `PriorityQueue` in de vorm van een string met op elke afzonderlijke regel: het prioriteitsgetal, gevolgd door een : en daarachter het element zelf.
  Voorbeeld voor een PriorityQueue van strings (uiteraard gesorteerd volgens prioriteit):

```
5: Denver
3: Oslo
2: Tokio
2: Rio
```

2.5. Maak nu een aparte klasse `Demo_2` met daarin een `main`.
Test eerst met een `PriorityQueue` van strings. Neem deze testcode over:

```
PriorityQueue<String> myQueue = new PriorityQueue<>();
myQueue.enqueue("Tokio", 2);
        myQueue.enqueue("Denver", 5);
        myQueue.enqueue("Rio", 2);
        myQueue.enqueue("Oslo", 3);
System.out.println("Overzicht van de PriorityQueue:");
System.out.println(myQueue.toString());
System.out.println("aantal: " + myQueue.getSize());
System.out.println("positie van Tokio: " + myQueue.search("Tokio"));
System.out.println("positie van Nairobi: " + myQueue.search("Nairobi"));
for(int i = 0; i < 4; i++) {
    System.out.println("Dequeue: " + myQueue.dequeue());
}
System.out.println("Size na dequeue: " + myQueue.getSize());
```

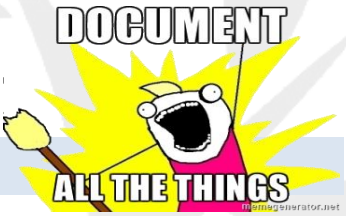
2.6. Verwachte output:

```
Overzicht van de PriorityQueue:
5: Denver
3: Oslo
2: Tokio
2: Rio

aantal: 4
positie van Tokio: 3
positie van Nairobi: -1
Dequeue: Denver
Dequeue: Oslo
Dequeue: Tokio
Dequeue: Rio
Size na dequeue: 0
```

2.7. Maak nu een `PriorityQueue` met elementen van de basisklasse (In ons geval dus een `PriorityQueue` van `Dictator`-objecten). Gebruik de klasse `Data` om elementen aan de `PriorityQueue` toe te voegen en geef telkens als prioriteit een randomwaarde mee (1 tem 5). Probeer alle functionaliteiten uit!

2.8. Vervang in de `main` van `Demo_2` alle declaraties door het keyword **`var`**.
Controleer of de applicatie gerund kan worden en dezelfde output geeft.



3. Javadoc

3.1. Schrijf **Javadoc**-commentaar voor de basisklasse (in ons geval: `Dictator`):

- Schrijf een class-header met een korte beschrijving en een `@author` en een `@version`-tag
- Voor zie in de class-header ook een `@see`-tag met een link naar meer informatie over je onderwerp. In ons voorbeeld linken we door naar een Wikipedia-pagina met meer uitleg over dictators: https://nl.wikipedia.org/wiki/Dictator_(modern)
- Schrijf boven elke contructor en elke methode gepaste commentaar. Voor een property (attribuut met een simpele getter/setter) mag je in de plaats ook het attribuut documenteren. Merk op dat IntelliJ zelf de gepaste Javadoc-tags voorziet.

- Eén van de attributen in de basisklasse is een enum. Voorzie bij de getter en setter van dit attribuut een verwijzing naar deze enum-klasse (gebruik `@see`)

3.2. Ga naar de menukeuze "*Tools* > *Generate JavaDoc*" en maak documentatie **alleen** voor de module `2_generics`.
Plaats de HTML-documenten in een nieuwe submap **doc** onder je project. Controleer het resultaat in een webbrowser →



Gebruik het *Terminal* venster:

- `gradlew run` om alle modules te runnen (dus zowel `m1_herhaling` als `m2_generics`)
- `gradlew m2_generics:run` om enkel deze module te runnen
- `git status -u` om te controleren of ook de javadoc-files worden meegenomen naar git
- `git add`, `git commit -m "..."` en `git push`
- Controleer jouw repo op GitLab: kloppen alle wijzigingen? Staat ook de javadoc op GitLab?